Guía de instalación del laboratorio SQLi

Requisitos, configuración y puesta en marcha de sqli-vulnerable-lab

Documento	INSTALLATION_GUIDE.pdf
Proyecto	sqli-vulnerable-lab (Flask + PyMySQL + MariaDB)
Versión del documento	1.0
Fecha	7 de noviembre de 2026
Elaborado por	Technical Writing + Security Engineering
Clasificación	Uso educativo interno - datos ficticios
Base del contenido	Código del repositorio: app/, database/, docker-compose.yml, Dockerfile, scripts/, tests/, docs/seguridad.md

Advertencia: Este laboratorio es intencionalmente vulnerable y esta destinado exclusivamente a la formación en entornos locales y controlados. No lo despliegues en redes publicas ni uses sus técnicas contra sistemas sin autorización expresa. Todos los datos son ficticios.

Índice

1. Sobre este documento . . . 3
2. Requisitos previos . . . 3
3. Instalación de Docker . . . 3
3.1 macOS . . . 3
3.2 Linux (Debian / Ubuntu) . . . 4
3.3 Windows . . . 4
4. Estructura del proyecto . . . 4
5. Configuración . . . 6
5.1 Variables de entorno . . . 6
5.2 docker-compose.yml . . . 6
5.3 Dockerfile y arranque del contenedor . . . 8
6. Instalación y puesta en marcha . . . 8
7. Inicio de la máquina y estado de los servicios . . . 9
8. Acceso al laboratorio . . . 10
8.1 Credenciales ficticias . . . 10
8.2 Flujo de uso habitual . . . 10
9. Pruebas incluidas . . . 11
10. Solución de problemas . . . 11
11. Mantenimiento y apagado . . . 11
12. Anexo: comandos rápidos . . . 12

1. Sobre este documento

Este manual describe como instalar, configurar y poner en marcha el laboratorio educativo **sqli-vulnerable-lab**: una aplicación Flask con PyMySQL y MariaDB, empaquetada con Docker Compose, que contiene vulnerabilidades de inyección SQL **intencionadas** con fines de formación. El contenido se ha elaborado a partir del código real del repositorio (*app/*, *database/*, *docker-compose.yml*, *Dockerfile*, *scripts/*, *tests/*, *docs/seguridad.md*).

> **Advertencia:** La aplicación es intencionalmente vulnerable. Ejecutala únicamente en local (127.0.0.1) o en una red aislada, y no la uses contra sistemas sin autorización expresa. Todos los datos son ficticios.

El documento esta dirigido a perfiles técnicos que necesitan levantar el entorno de forma reproducible: estudiantes, formadores, auditores y personal de plataformas. No se asumen conocimientos previos de Docker, aunque si se manejar la línea de comandos.

2. Requisitos previos

Los requisitos se han tomado del README del proyecto y del Dockerfile. Docker 29+ y Docker Compose v2+ son obligatorios; Python 3.12+ solo es necesario si se desea ejecutar las pruebas locales.

Requisito	Versión / mínimo	Comentario
Docker Engine	29 o superior	Motor de contenedores.
Docker Compose	v2 o superior (plugin)	Orquestación de los servicios db y web.
Python	3.12 o superior	Solo para las pruebas con pytest y requests (entorno local).
Sistema operativo	macOS, Linux o Windows (WSL2)	Cualquier plataforma con Docker Desktop o Docker Engine.
Memoria RAM	4 GB recomendados	MariaDB y gunicorn conviven en el mismo equipo.
Espacio en disco	5 GB libres	Imagen python:3.12-slim, mariadb:11.4 y dependencias.
Acceso a red	Solo descarga inicial de imágenes	El laboratorio no requiere salida a Internet para funcionar.

Comprobación rápida de versiones (en una terminal):

```bash
docker --version
docker compose version
python3 --version
```

Si alguno de los comandos no esta disponible, continua por la sección **3. Instalación de Docker** antes de seguir.

3. Instalación de Docker

Los pasos varian segun el sistema operativo. En todos los casos es necesario que el servicio de Docker este arrancado antes de ejecutar cualquier comando *docker*.

3.1 macOS

- Opción recomendada: instalar **Docker Desktop para Mac** desde *https://www.docker.com/products/docker-desktop/*. Descarga el instalador para el chip de tu equipo (Intel o Apple Silicon) y arrastra la aplicación a /Applications.
- Opción alternativa con Homebrew: *brew install --cask docker*. Después, abre Docker Desktop desde Aplicaciones para arrancar el servicio.
- Verificación: *docker --versión* y *docker run --rm hello-world*.

3.2 Linux (Debian / Ubuntu)

bash

```
# Docker Engine + plugin Compose (repositorio oficial)
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu
$(. /etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
# Permitir usar docker sin sudo (requiere volver a iniciar sesion)
sudo usermod -aG docker $USER
```

En distribuciones basadas en Fedora/RHEL sustituye *apt-get* por *dnf* y sigue la documentación oficial de Docker Engine para tu distribución.

3.3 Windows

- Instala **Docker Desktop para Windows** con el backend WSL2 (requisito: Windows 10/11 de 64 bits y virtualización activada en BIOS).
- Durante la instalación marca la opción *Use WSL 2 instead of Hyper-V*.
- Verificación en PowerShell: *docker --versión* y *docker compose versión*.

Verificación obligatoria

Ejecuta *docker run --rm hello-world*. Si la salida contiene *Hello from Docker!*, el motor funciona correctamente.

4. Estructura del proyecto

El repositorio es autocontenido: el código de la aplicación, el esquema y los datos de ejemplo, los scripts de gestión y las pruebas. La estructura real es la siguiente:

arbol de directorios

```
sqli-vulnerable-lab/
|-- app/
| |-- __init__.py # factory create_app() y configuracion por entorno
| |-- db.py # query_raw() (vulnerable) y query_parametrized() (seguro)
| |-- routes/
| | |-- auth.py # /login, /secure-login, /logout
| | |-- dashboard.py # /dashboard (requiere sesion)
| | |-- scenarios.py # /search y /product
| |-- static/
| | |-- css/style.css
| | |-- js/main.js
| |-- templates/
| |-- base.html, login.html, secure_login.html
| |-- dashboard.html, search.html, product.html, results.html
|-- database/
| |-- init.sql # esquema: roles, users, activity_logs, products, customers
| |-- seed.sql # datos ficticios (usuarios, productos, clientes, logs)
|-- scripts/
| |-- start.sh # docker compose build + up -d
| |-- stop.sh # docker compose down (conserva el volumen)
| |-- reset_db.sh # down -v y reinicializa esquema + seed
| |-- cleanup.sh # down -v --rmi all
| |-- logs.sh # docker compose logs -f
|-- tests/
| |-- conftest.py # fixture base_url (LAB_URL)
| |-- test_login_vulnerable.py
| |-- test_login_secure.py
| |-- test_sqli_scenarios.py
| |-- test_docker.py
|-- docs/
| |-- seguridad.md # notas de seguridad y reglas de uso
|-- docker-compose.yml # servicios db (mariadb:11.4) y web (gunicorn)
|-- Dockerfile # python:3.12-slim + requirements.txt
|-- docker-entrypoint.sh # espera a la DB antes de arrancar gunicorn
|-- requirements.txt # Flask 3.1.0, PyMySQL 1.1.1, gunicorn 23.0.0
|-- .env.example # Variables documentadas (valores ficticios)
|-- .gitignore
|-- README.md
```

Ruta	Contenido
app/__init__.py	Factory create_app(): configuración por variables de entorno y registro de los blueprints auth, dashboard y scenarios.
app/db.py	Capa de acceso a datos: query_raw() (SQL concatenado, vulnerable) y query_parametrized() (marcadores %s, seguro).
app/routes/auth.py	Rutas /login (vulnerable), /secure-login (parametrizado) y /logout.
app/routes/scenarios.py	Rutas /search (error-based) y /product (boolean-based y time-based).
app/routes/dashboard.py	Ruta /dashboard protegida por sesión.
database/init.sql	Esquema: roles, users, activity_logs, products y customers. Se ejecuta al crear el volumen.
database/seed.sql	Datos ficticios: 3 roles, 8 usuarios, 6 productos, 7 clientes y 8 registros de actividad.
scripts/	start.sh, stop.sh, reset_db.sh, cleanup.sh y logs.sh.
tests/	Pruebas pytest + requests: login vulnerable, login seguro, escenarios SQLi e infraestructura Docker.
docker-compose.yml	Servicios db (mariadb:11.4) y web (build del Dockerfile, gunicorn). Red bridge sqli-net y volumen db-data.
Dockerfile	Imagen python:3.12-slim con Flask 3.1.0, PyMySQL 1.1.1 y gunicorn 23.0.0; expone el puerto 8080.
docker-entrypoint.sh	Espera activa a la base de datos (hasta 60 intentos, 2 s) antes de arrancar gunicorn.
.env.example	Variables documentadas con valores ficticios.

5. Configuración

5.1 Variables de entorno

La aplicación lee la configuración de las variables de entorno en *app/__init__.py* (factory) y en *app/db.py* (conexiones fuera de contexto Flask). Los valores del despliegue se definen directamente en *docker-compose.yml*; el archivo *.env.example* documenta las mismas variables para referencia y para las pruebas. **Todos los valores son ficticios** y solo sirven al laboratorio.

Variable	Valor en docker-compose.yml	Descripción
DB_HOST	db	Nombre del servicio de base de datos en la red Docker.
DB_PORT	3306	Puerto interno de MariaDB (no publicado al host).
DB_NAME	sqli_lab	Base de datos creada por init.sql.
DB_USER	lab_user	Usuario de aplicación (privilegios limitados).
DB_PASSWORD	lab_pass_ficticio_cambiar	Contraseña ficticia del usuario de aplicación (definida en docker-compose.yml).
MARIADB_ROOT_PASSWORD	root_ficticio_cambiar	Contraseña ficticia de root del contenedor MariaDB (solo administración local).
SECRET_KEY	secreto_ficticio_cambiar_en_produccion	Clave usada por Flask para firmar la cookie de sesión.
LAB_URL	http://127.0.0.1:8080	URL base usada únicamente por las pruebas (tests/conftest.py).

Información: Los valores **_ficticio_cambiar* son deliberadamente obvios: cambialos si reutilizas la configuración como plantilla. En el laboratorio educativo no se manejan secretos reales.

Para trabajar con un archivo *.env* local (opcional, no lo usa el Compose del laboratorio):

bash

```
cp .env.example .env
# ajusta los valores y conserva el archivo fuera del control de versiones
```

El repositorio ignora *.env* y *venv/* en *.gitignore*; no subas credenciales reales ni entornos virtuales.

5.2 docker-compose.yml

Define dos servicios sobre una red bridge llamada *sqli-net* y un volumen nominado *db-data*. El contenido real es:

yaml

```
services:
db:
image: mariadb:11.4
container_name: sqli-lab-db
restart: unless-stopped
environment:
MARIADB_ROOT_PASSWORD: root_ficticio_cambiar
MARIADB_DATABASE: sqli_lab
MARIADB_USER: lab_user
MARIADB_PASSWORD: lab_pass_ficticio_cambiar
volumes:
- db-data:/var/lib/mysql
- ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
- ./database/seed.sql:/docker-entrypoint-initdb.d/seed.sql:ro
networks:
- sqli-net
healthcheck:
test: ["CMD", "mariadb-admin", "ping", "-h", "localhost", "-u", "root",
"-proot_ficticio_cambiar"]
interval: 5s
timeout: 5s
retries: 20
start_period: 30s
# Sin puertos publicados: la base de datos solo es accesible en la red interna.
web:
build: .
container_name: sqli-lab-web
restart: unless-stopped
depends_on:
db:
condition: service_healthy
environment:
DB_HOST: db
DB_PORT: "3306"
DB_NAME: sqli_lab
DB_USER: lab_user
DB_PASSWORD: lab_pass_ficticio_cambiar
SECRET_KEY: secreto_ficticio_cambiar_en_produccion
ports:
- "127.0.0.1:8080:8080"
networks:
- sqli-net
networks:
sqli-net:
driver: bridge
volumes:
db-data:
```

- **db**: imagen mariadb:11.4; crea la base *sqli_lab* y el usuario *lab_user*; monta init.sql y seed.sql en /docker-entrypoint-initdb.d (se ejecutan solo la primera vez que se crea el volumen); healthcheck con mariadb-admin ping cada 5 s.
- **web**: construye la imagen desde el Dockerfile; depende de que db este *service_healthy*; publica únicamente 127.0.0.1:8080->8080/tcp.
- **Red y volumen**: red bridge *sqli-net* y volumen *db-data* para persistencia de la base de datos.

5.3 Dockerfile y arranque del contenedor

dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
RUN chmod +x /usr/local/bin/docker-entrypoint.sh
EXPOSE 8080
ENTRYPOINT ["docker-entrypoint.sh"]
CMD ["gunicorn", "--bind", "0.0.0.0:8080", "--workers", "2", "--timeout", "60", "app:app"]
```

El entrypoint espera a que MariaDB responda antes de ejecutar gunicorn, con un máximo de 60 intentos separados por 2 segundos. Su contenido real es:

shell

```
#!/bin/sh
# Entrypoint: espera a que MariaDB este disponible antes de arrancar gunicorn.
set -e
DB_HOST="${DB_HOST:-db}"
DB_PORT="${DB_PORT:-3306}"
DB_USER="${DB_USER:-lab_user}"
DB_PASSWORD="${DB_PASSWORD:-lab_pass_ficticio_cambiar}"
export DB_HOST DB_PORT DB_USER DB_PASSWORD
echo "[entrypoint] Esperando a la base de datos en ${DB_HOST}:${DB_PORT}..."
attempt=0
until python -c "
import os
import pymysql
pymysql.connect(
host=os.environ['DB_HOST'],
port=int(os.environ['DB_PORT']),
user=os.environ['DB_USER'],
password=os.environ['DB_PASSWORD'],
connect_timeout=3,
).close()
" 2>/dev/null; do
attempt=$((attempt + 1))
if [ "$attempt" -ge 60 ]; then
echo "[entrypoint] Tiempo de espera agotado." >&2
exit 1
fi
echo "[entrypoint] Base de datos no disponible, reintentando en 2s..."
sleep 2
done
echo "[entrypoint] Base de datos disponible. Iniciando la aplicacion."
exec "$@"
```

Gunicorn arranca con *--workers 2* y *--timeout 60* enlazado a *0.0.0.0:8080* dentro del contenedor; la publicación al host se limita a 127.0.0.1.

6. Instalación y puesta en marcha

Desde la raíz del proyecto, el script de inicio construye las imágenes y levanta los servicios en segundo plano:

shell

```
#!/usr/bin/env bash
# Construye y arranca el laboratorio (db + web) en segundo plano.
set -euo pipefail
cd "$(dirname "$0")/.."
echo "[start] Construyendo imagenes y levantando servicios..."
docker compose build
docker compose up -d
echo "[start] Laboratorio iniciado. Acceso: http://127.0.0.1:8080"
```

Ejecución paso a paso:

bash

```
cd /Users/ing_juan/Desktop/maquinas_virtuales/sqli-vulnerable-lab
./scripts/start.sh
```

El script ejecuta *docker compose build* y *docker compose up -d*. Equivalentes manuales, si prefieres ejecutarlos por separado:

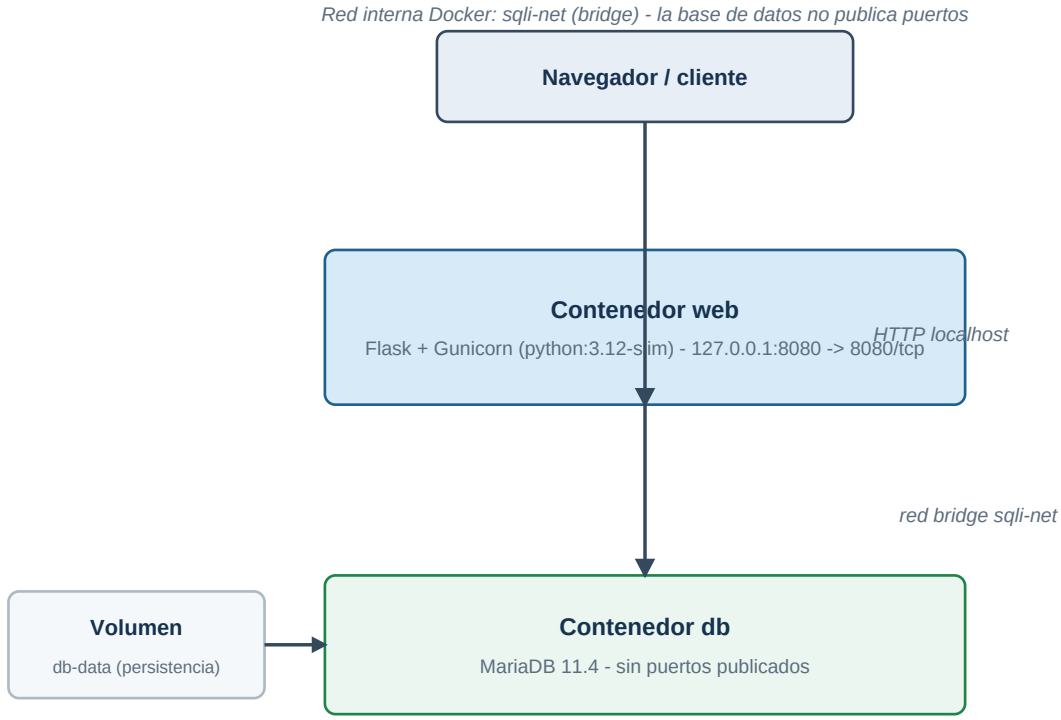
bash

```
docker compose build # construye la imagen del servicio web
docker compose up -d # crea y arranca db y web en segundo plano
docker compose ps # estado de los contenedores
```

Durante el primer arranque el servicio *web* espera a que el healthcheck de *db* sea *healthy*. En condiciones normales el proceso completo tarda entre 30 y 90 segundos, segun el equipo.

7. Inicio de la máquina y estado de los servicios

La siguiente figura resume la arquitectura de contenedores y el aislamiento de la base de datos:



Una vez ejecutado start.sh, el estado esperado de los contenedores es:

Servicio	Contenedor	Imagen	Puertos publicados
db	sqli-lab-db	mariadb:11.4	Ninguno (3306/tcp solo en la red interna sqli-net)
web	sqli-lab-web	build del Dockerfile	127.0.0.1:8080->8080/tcp

Comandos de verificación de estado:

```bash
docker compose ps # estado y puertos
docker compose logs -f # logs en tiempo real (Ctrl+C para salir)
docker compose logs db # solo la base de datos
docker inspect sqli-lab-db --format '{{.State.Health.Status}}'
```

El campo *Puertos* de *db* muestra *3306/tcp* sin ningun mapeo ->: eso confirma que la base de datos no publica puertos en el host. Esta comprobación forma parte de las pruebas *test_docker.py*.

8. Acceso al laboratorio

Con los servicios en marcha, la aplicación responde en **http://127.0.0.1:8080**. El enlace esta restringido a la interfaz de bucle (loopback): no es accesible desde otros equipos de la red.

Ruta	Metodo	Descripción
/login	GET / POST	Autenticación vulnerable: concatena usuario y contraseña en la consulta SQL.
/secure-login	GET / POST	Autenticación segura: usa consulta parametrizada con marcadores %s.
/logout	GET	Cierra la sesión (session.clear()) y redirige a /login.
/dashboard	GET	Panel con usuarios y actividad. Requiere sesión; si no hay, redirige a /login.
/search	GET	Buscador de productos (error-based). Parámetro: q.
/product	GET	Producto por ID (boolean-based y time-based). Parámetro: id.

Comprobación rápida con curl:

```bash
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080/login
# salida esperada: 200
```

8.1 Credenciales ficticias

El seed inserta ocho usuarios con la misma contraseña ficticia (*Password123!*), que esta publicada en el README del proyecto y en los formularios de demostración. Son credenciales de laboratorio, no secretos reales.

Usuario	Email	Rol	Uso en el laboratorio
admin	admin@lab.local	admin	Administrador del laboratorio
jdoe	jdoe@lab.local	user	Usuario estandar
asmith	asmith@lab.local	user	Usuario estandar
mbrown	mbrown@lab.local	auditor	Auditor de actividad
lgarcia	lgarcia@lab.local	user	Usuario estandar
kwilson	kwilson@lab.local	user	Usuario estandar
nperez	nperez@lab.local	user	Usuario estandar
slee	slee@lab.local	user	Usuario estandar

8.2 Flujo de uso habitual

- Abre *http://127.0.0.1:8080/login* y autentificate con *admin* / *Password123!* para acceder al dashboard.
- Repite el inicio de sesión en */secure-login* para comparar el comportamiento parametrizado.
- Prueba los escenarios de inyección en */search* y */product*; las propias páginas incluyen botones con payloads de demostración.
- Cierra la sesión con */logout*.

9. Pruebas incluidas

El proyecto incluye pruebas pytest que contactan con la aplicación en ejecución. Asumen que los servicios estan levantados y usan la variable *LAB_URL* (por defecto *http://127.0.0.1:8080*).

bash

```
python3 -m venv venv
source venv/bin/activate
pip install pytest requests
pytest -v
```

Archivo de pruebas	Que verifica
tests/test_login_vulnerable.py	Login normal (302 a /dashboard) y bypass clasico con admin' OR '1'='1' -- -.
tests/test_login_secure.py	Login normal y bloqueo del bypass en /secure-login (200 con error, sin sesión).
tests/test_sqli_scenarios.py	Error visible en /search, diferencia booleana en /product y retardo SLEEP(5) > 4 s.
tests/test_docker.py	Contenedores activos, db sin puertos publicados y web solo en 127.0.0.1:8080.

10. Solución de problemas

Síntoma	Causa probable	Solución
Cannot connect to the Docker daemon	Docker Desktop / servicio Docker no arrancado.	Inicia Docker Desktop (o sudo systemctl start docker) y reintenta.
port is already allocated (8080)	Otro proceso del host usa el puerto 8080.	Localizalo con lsof -iTCP:8080 -sTCP:LISTEN y detenlo, o cambia el mapeo en docker-compose.yml (no recomendado).
El contenedor web se reinicia o no responde	La base de datos todavía no esta healthy.	Consulta docker compose logs db y espera al healthcheck (interval 5 s, retries 20, start_period 30 s).
El entrypoint muestra "Tiempo de espera agotado"	MariaDB no respondio en 60 intentos (aprox. 2 minutos).	Revisa los logs de db, verifica el volumen y ejecuta ./scripts/reset_db.sh.
Errores de conexión intermitentes o esquema corrupto	Volumen db-data en un estado inconsistente.	Ejecuta ./scripts/reset_db.sh para recrear el volumen y reaplicar init.sql + seed.sql.
Los cambios en el código no se reflejan	Imagen construida anteriormente.	Ejecuta ./scripts/reset_db.sh (incluye --build) o docker compose build --no-cache.
La base de datos no arranca tras un reinicio	MySQL todavía inicializandose.	Espera a que el healthcheck pase a healthy; el servicio web se reintenta con restart: unless-stopped.

Información: Si el problema persiste, ejecuta *./scripts/logs.sh* y revisa los mensajes de ambos servicios antes de reinstalar Docker.

11. Mantenimiento y apagado

Script	Efecto	Cuando usarlo
scripts/start.sh	docker compose build + up -d	Puesta en marcha normal.
scripts/stop.sh	docker compose down (conserva el volumen)	Detener el laboratorio sin perder datos.
scripts/reset_db.sh	down -v y up -d --build	Reinicializar esquema y datos ficticios.
scripts/cleanup.sh	down -v --rmi all	Limpieza total: contenedores, volumen e imágenes.
scripts/logs.sh	docker compose logs -f	Diagnostico en tiempo real.

12. Anexo: comandos rápidos

bash

```
# Arrancar y comprobar
./scripts/start.sh
docker compose ps
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080/login
# Ver logs y reiniciar la base de datos
./scripts/logs.sh
./scripts/reset_db.sh
# Detener y limpiar
./scripts/stop.sh
./scripts/cleanup.sh
```

Documentación relacionada

Complementa esta guía con *SQL_INJECTION_LAB_GUIDE.pdf* (uso didáctico de los escenarios), *SECURITY_ASSESSMENT_REPORT.pdf* (análisis de seguridad) y *SQL_INJECTION_REMEDIATION_GUIDE.pdf* (medidas correctivas).